ASSIGNMENT-15.1

HT No:2303A51781

Batch No:28

Course:AI Assisted Coding

Task 1 :Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

• Endpoints required:

o GET /students → List all students o POST /students →

Add a new student o PUT /students/{id} → Update

student details o DELETE /students/{id} → Delete a

student record • Use an in-memory data structure (list

or dictionary) to store records.

• Ensure API responses are in JSON format

Prompt:

Generate a RESTful API using Python Flask for managing student records.

The API should support CRUD operations:

- GET /students → retrieve all students

- POST /students → add a new student

- PUT /students/{id} → update student details

- DELETE /students/{id} → delete a student

Use an in-memory dictionary or list to store data.

Ensure all responses are in JSON format.

Add proper error handling and comments.

Python Code:

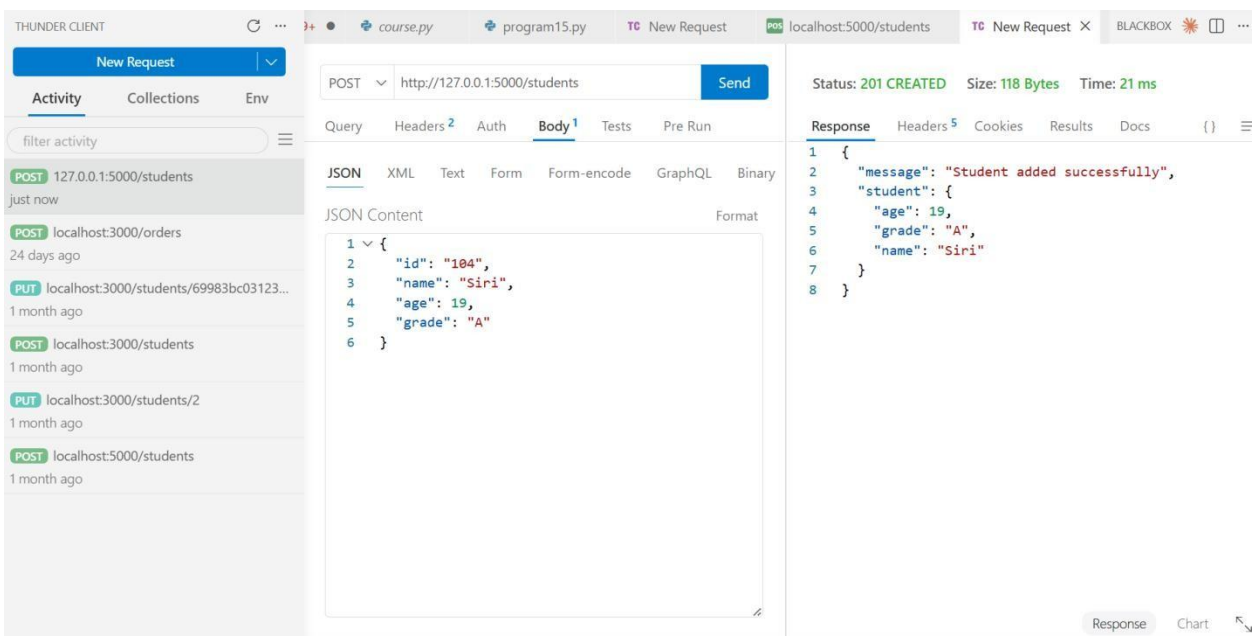
```python
#Generate a RESTful API using Python Flask for managing student records.
#The API should support CRUD operations:
#- GET /students → retrieve all students
#- POST /students → add a new student
#- PUT /students/{id} → update student details
#- DELETE /students/{id} → delete a student
#Use an in-memory dictionary or list to store data.Ensure all responses are in JSON format.Add proper error handling and comments.
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage for students
students = {}


@app.route('/students', methods=['GET'])
def get_students():
    """Retrieve all students."""
    return jsonify({
        "success": True,
        "data": students
    }), 200


@app.route('/students', methods=['POST'])
def add_student():
    """Add a new student."""
    try:
        data = request.get_json()
```

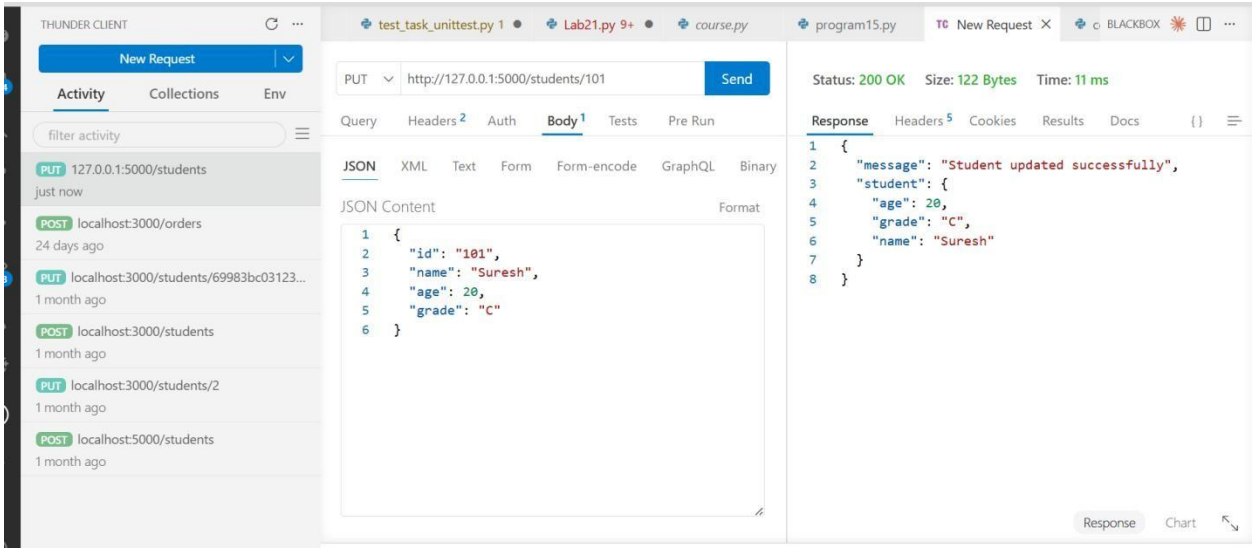
```python
try:
    data = request.get_json()

    # Validate input
    if not data:
        return jsonify({"error": "Request body must be JSON"}), 400

    student_id = data.get('id')
    if not student_id:
        return jsonify({"error": "Student ID is required"}), 400

    if student_id in students:
        return jsonify({"error": "Student already exists"}), 409

    # Create new student
    students[student_id] = {
        "name": data.get('name'),
        "age": data.get('age'),
        "grade": data.get('grade')
    }

    return jsonify({
        "message": "Student added successfully",
        "student": students[student_id]
    }), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500
```

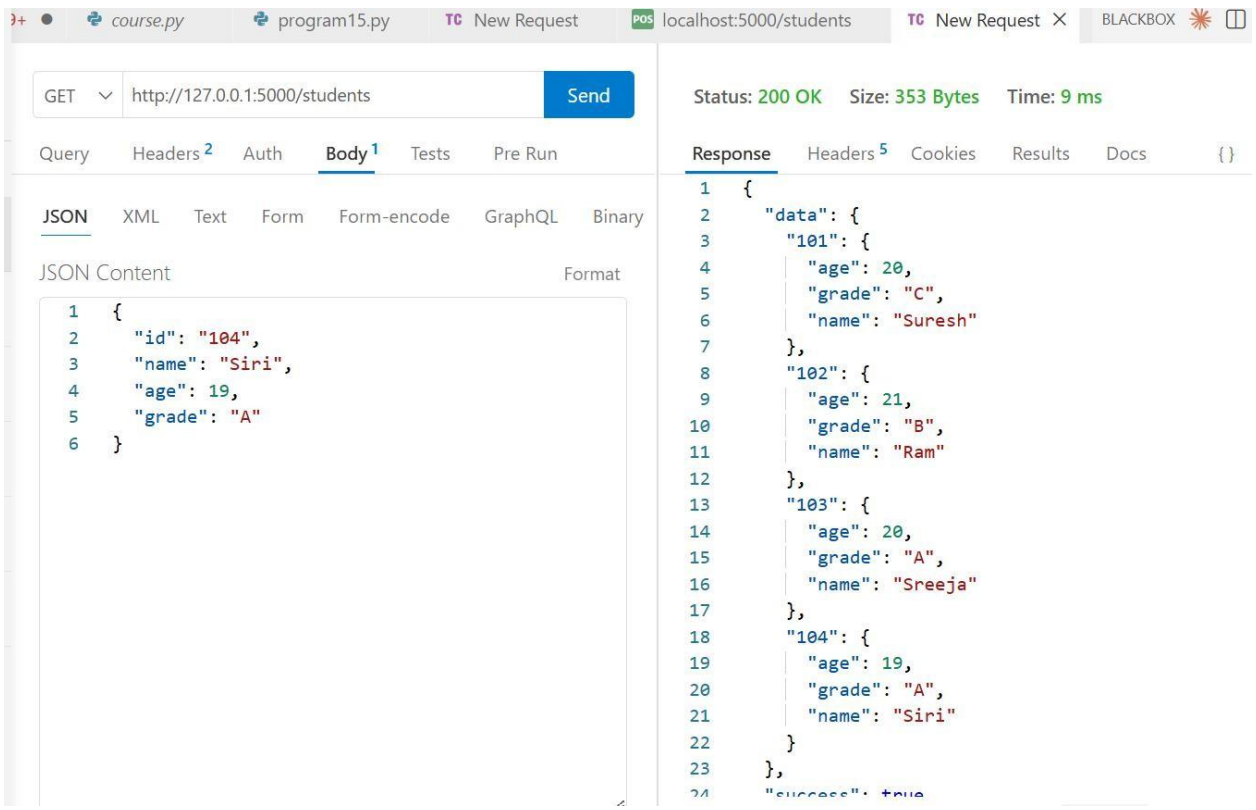
```python
@app.route('/students/<student_id>', methods=['PUT'])
def update_student(student_id):
    """Update student details."""
    try:
        if student_id not in students:
            return jsonify({"error": "Student not found"}), 404

        data = request.get_json()
        if not data:
            return jsonify({"error": "Request body must be JSON"}), 400

        # Update only provided fields
        students[student_id].update({
            "name": data.get('name', students[student_id]['name']),
            "age": data.get('age', students[student_id]['age']),
            "grade": data.get('grade', students[student_id]['grade'])
        })

        return jsonify({
            "message": "Student updated successfully",
            "student": students[student_id]
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500


@app.route('/students/<student_id>', methods=['DELETE'])
```

```python
@app.route('/students/<student_id>', methods=['DELETE'])
def delete_student(student_id):
    """Delete a student."""
    try:
        if student_id not in students:
            return jsonify({"error": "Student not found"}), 404

        deleted_student = students.pop(student_id)

        return jsonify({
            "message": "Student deleted successfully",
            "student": deleted_student
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500


# Run the Flask app
if __name__ == '__main__':
    app.run(debug=True)
```

REST API Methods Documentation:




THUNDER CLIENT
New Request
Activity
Collections
Env
filter activity
PUT 127.0.0.1:5000/students
just now
POST localhost:3000/orders
24 days ago
PUT localhost:3000/students/69983bc03123...
1 month ago
POST localhost:3000/students
1 month ago
PUT localhost:3000/students/2
1 month ago
POST localhost:5000/students
1 month ago
test_task_unittest.py 1
Lab21.py 9+
course.py
program15.py
New Request
BLACKBOX
PUT
http://127.0.0.1:5000/students/101
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"id": "101",
"name": "Suresh",
"age": 20,
"grade": "C"
}
Status: 200 OK
Size: 122 Bytes
Time: 11 ms
Response
Headers 5
Cookies
Results
Docs
{
"message": "Student updated successfully",
"student": {
"age": 20,
"grade": "C",
"name": "Suresh"
}
}
Response
Chart


course.py
program15.py
New Request
localhost:5000/students
New Request
BLACKBOX
GET
http://127.0.0.1:5000/students
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"id": "104",
"name": "Siri",
"age": 19,
"grade": "A"
}
Status: 200 OK
Size: 353 Bytes
Time: 9 ms
Response
Headers 5
Cookies
Results
Docs
{
"data": {
"101": {
"age": 20,
"grade": "C",
"name": "Suresh"
},
"102": {
"age": 21,
"grade": "B",
"name": "Ram"
},
"103": {
"age": 20,
"grade": "A",
"name": "Sreeja"
},
"104": {
"age": 19,
"grade": "A",
"name": "Siri"
}
},

Output:

Pretty-print


```
{
  "data": {
    "101": {
      "age": 20,
      "grade": "C",
      "name": "Suresh"
    },
    "102": {
      "age": 21,
      "grade": "B",
      "name": "Ram"
    },
    "103": {
      "age": 20,
      "grade": "A",
      "name": "Sreeja"
    },
    "104": {
      "age": 19,
      "grade": "A",
      "name": "Siri"
    }
  },
  "success": true
}
```


DELETE http://127.0.0.1:5000/students/104 Send

Status: 200 OK Size: 120 Bytes Time: 5 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format


```
{
  "id": "104",
  "name": "Siri",
  "age": 19,
  "grade": "A"
}
```


Response Headers 5 Cookies Results Docs


```
{
  "message": "Student deleted successfully",
  "student": {
    "age": 19,
    "grade": "A",
    "name": "Siri"
  }
}
```


After Deletion Output:

Pretty-print


```
{
  "data": {
    "101": {
      "age": 20,
      "grade": "C",
      "name": "Suresh"
    },
    "102": {
      "age": 21,
      "grade": "B",
      "name": "Ram"
    },
    "103": {
      "age": 20,
      "grade": "A",
      "name": "Sreeja"
    }
  },
  "success": true
}
```


Justification:

1. Provides a CRUD interface to manage student information efficiently.
2. Uses in-memory storage (list/dictionary) for quick prototyping.
3. JSON responses ensure compatibility with web or mobile clients.
4. RESTful endpoints allow easy integration with front-end apps.
5. AI assists in generating code, test cases, and validation logic.

Task 2 : Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

• Endpoints required:

o GET /books → Retrieve all books

o POST /books → Add a new book

o GET /books/{id} → Get details of a specific book

o PATCH /books/{id} → Update partial book

details

(e.g., availability)

o DELETE /books/{id} → Remove a book •

Implement error handling for invalid requests.

Prompt:

Generate a RESTful API using Python Flask for a Library Book Management system.

Requirements:

- GET /books → retrieve all books

- POST /books → add a new book

- GET /books/{id} → get book details by ID

- PATCH /books/{id} → partially update book details (like availability)

- DELETE /books/{id} → delete a book

Use an in-memory dictionary to store books.

Add proper error handling for invalid requests.

Return all responses in JSON format.

Python Code:

p15_2.py > update_book

```python
#Generate a RESTful API using Python Flask for a Library Book Management system.
#Requirements:
#- GET /books → retrieve all books
#- POST /books → add a new book
#- GET /books/{id} → get book details by ID
#- PATCH /books/{id} → partially update book details (like availability)
#- DELETE /books/{id} → delete a book
#Use an in-memory dictionary to store books.
#Add proper error handling for invalid requests.
#Return all responses in JSON format.
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for books
books = {}
@app.route('/books', methods=['GET'])
def get_books():
    """Retrieve all books."""
    return jsonify({
        "success": True,
        "data": books
    }), 200
@app.route('/books', methods=['POST'])
def add_book():
    """Add a new book."""
    try:
```

```python
def add_book():
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "Request body must be JSON"}), 400
        book_id = data.get('id')
        if not book_id:
            return jsonify({"error": "Book ID is required"}), 400
        if book_id in books:
            return jsonify({"error": "Book already exists"}), 409
        books[book_id] = {
            "title": data.get('title'),
            "author": data.get('author'),
            "available": data.get('available', True)
        }
        return jsonify({
            "message": "Book added successfully",
            "book": books[book_id]
        }), 201
    except Exception as e:
        return jsonify({"error": str(e)}), 500
@app.route('/books/<book_id>', methods=['GET'])
def get_book(book_id):
    """Get book details by ID."""
    book = books.get(book_id)
```

```python
def get_book(book_id):
    if not book:
        return jsonify({"error": "Book not found"}), 404
    return jsonify({
        "success": True,
        "data": book
    }), 200
@app.route('/books/<book_id>', methods=['PATCH'])
def update_book(book_id):
    """Partially update book details."""
    book = books.get(book_id)
    if not book:
        return jsonify({"error": "Book not found"}), 404
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "Request body must be JSON"}), 400
        # Update only provided fields
        for key in ['title', 'author', 'available']:
            if key in data:
                book[key] = data[key]
        return jsonify({
            "message": "Book updated successfully",
            "book": book
        }), 200
```

```python
    except Exception as e:
        return jsonify({"error": str(e)}), 500
@app.route('/books/<book_id>', methods=['DELETE'])
def delete_book(book_id):
    """Delete a book."""
    if book_id not in books:
        return jsonify({"error": "Book not found"}), 404
    del books[book_id]
    return jsonify({
        "message": "Book deleted successfully"
    }), 200
if __name__ == '__main__':
    app.run(debug=True)
```

REST API Methods Documentation:

POST http://127.0.0.1:5000/books Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "id": "101",
  "title": "Harry Potter",
  "author": "J.K. Rowling",
  "available": true
}
```

Status: 201 CREATED Size: 141 Bytes Time: 12 ms

Response Headers 5 Cookies Results Docs { }

```
{
  "book": {
    "author": "J.K. Rowling",
    "available": true,
    "title": "Harry Potter"
  },
  "message": "Book added successfully"
}
```

Response Chart

PROBLEMS 28 OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE Python

GET http://127.0.0.1:5000/books Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "id": "103",
  "title": "Server Busy",
  "author": "Jerry",
  "available": false
}
```

Status: 200 OK Size: 354 Bytes Time: 4 ms

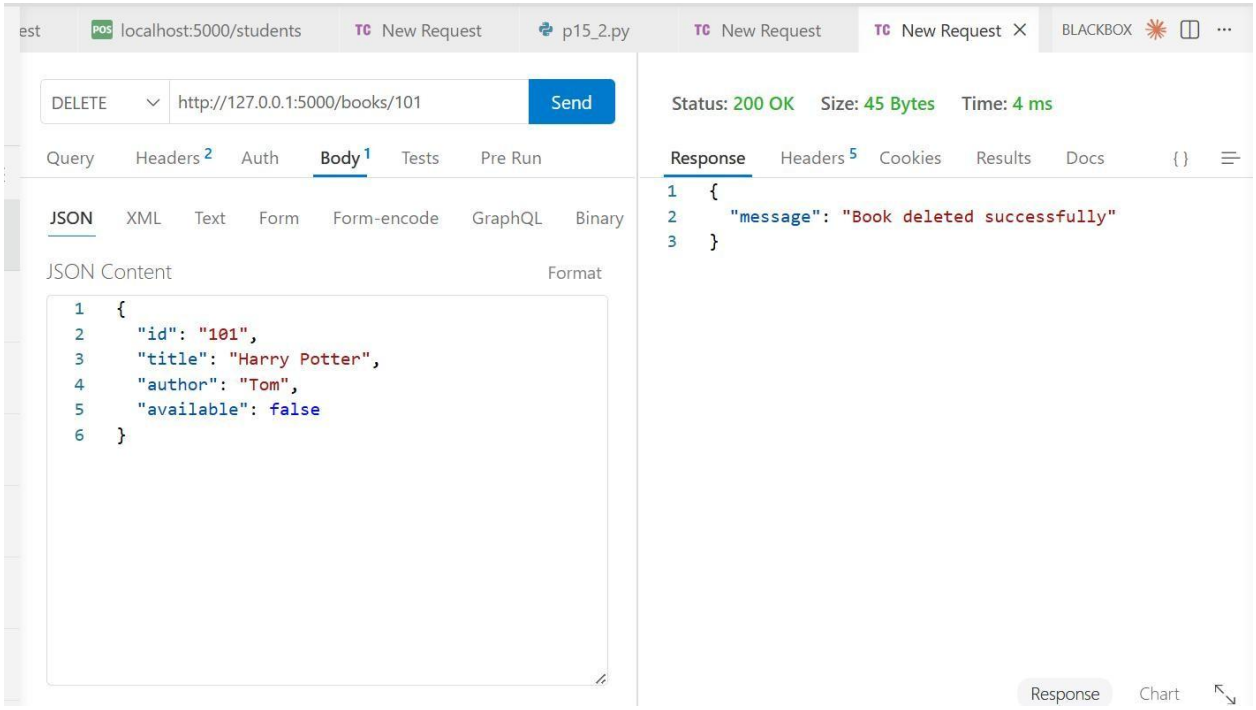
Response Headers 5 Cookies Results Docs { }

```
{
  "data": {
    "101": {
      "author": "J.K. Rowling",
      "available": true,
      "title": "Harry Potter"
    },
    "102": {
      "author": "John",
      "available": false,
      "title": "The route map of life"
    },
    "103": {
      "author": "Jerry",
      "available": false,
      "title": "Server Busy"
    }
  },
  "success": true
}
```

Response Chart

Pretty-print


```
{
  "data": {
    "101": {
      "author": "J.K. Rowling",
      "available": true,
      "title": "Harry Potter"
    },
    "102": {
      "author": "John",
      "available": false,
      "title": "The route map of life"
    },
    "103": {
      "author": "Jerry",
      "available": false,
      "title": "Server Busy"
    }
  },
  "success": true
}
```

After Deletion Output:

Pretty-print


```
{
  "data": {
    "102": {
      "author": "John",
      "available": false,
      "title": "The route map of life"
    },
    "103": {
      "author": "Jerry",
      "available": false,
      "title": "Server Busy"
    }
  },
  "success": true
}
```


Justification:

1. Enables management of library books with full CRUD support.
2. PATCH endpoint allows partial updates like book availability.
3. Error handling ensures robustness against invalid requests.
4. JSON responses standardize communication with other systems.
5. AI helps scaffold endpoints, suggest data validation, and create test cases.

Task 3 :Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

o GET /employees → List all employees o POST

/employees → Add a new employee with salary

details o PUT /employees/{id}/salary → Update

salary of an employee o DELETE /employees/{id}

→ Remove employee from system

• Use AI to:

o Suggest data model structure. o Add

comments/docstrings for all endpoints.

Prompt:

Generate a RESTful API using Python Flask for managing employee payroll.

Requirements:

- GET /employees → list all employees

- POST /employees → add a new employee with salary

- PUT /employees/{id}/salary → update salary of an employee

- DELETE /employees/{id} → remove an employee

Use an in-memory dictionary to store employee data.

Add clear comments/docstrings for all endpoints.

Ensure all responses are in JSON format.

Python Code:

```python
#Generate a RESTful API using Python Flask for managing employee payroll.
#Requirements:
#- GET /employees → list all employees
#- POST /employees → add a new employee with salary
#- PUT /employees/{id}/salary → update salary of an employee
#- DELETE /employees/{id} → remove an employee
#Use an in-memory dictionary to store employee data.Add clear comments/docstrings for all endpoints.Ensure all
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for employees
employees = {}
@app.route('/employees', methods=['GET'])
def get_employees():
    """List all employees."""
    return jsonify({
        "success": True,
        "data": employees
    }), 200
@app.route('/employees', methods=['POST'])
def add_employee():
    """Add a new employee with salary."""
    try:
        data = request.get_json()
        if not data:
```

```python
        data = request.get_json()
        if not data:
            return jsonify({"error": "Request body must be JSON"}), 400
        employee_id = data.get('id')
        if not employee_id:
            return jsonify({"error": "Employee ID is required"}), 400
        if employee_id in employees:
            return jsonify({"error": "Employee already exists"}), 409
        employees[employee_id] = {
            "name": data.get('name'),
            "salary": data.get('salary')
        }
        return jsonify({
            "message": "Employee added successfully",
            "employee": employees[employee_id]
        }), 201
    except Exception as e:
        return jsonify({"error": str(e)}), 500
@app.route('/employees/<employee_id>/salary', methods=['PUT'])
def update_salary(employee_id):
    """Update salary of an employee."""
    employee = employees.get(employee_id)
    if not employee:
```

```python
def update_salary(employee_id):
    if not employee:
        return jsonify({"error": "Employee not found"}), 404
    try:
        data = request.get_json()
        if not data or 'salary' not in data:
            return jsonify({"error": "Salary is required in JSON body"}), 400
        employee['salary'] = data['salary']
        return jsonify({
            "message": "Salary updated successfully",
            "employee": employee
        }), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
@app.route('/employees/<employee_id>', methods=['DELETE'])
def delete_employee(employee_id):
    """Remove an employee."""
    if employee_id not in employees:
        return jsonify({"error": "Employee not found"}), 404
    del employees[employee_id]
    return jsonify({
        "message": "Employee removed successfully"
    }), 200
if __name__ == '__main__':
```

```python
def delete_employee(employee_id):
    if employee_id not in employees:
        return jsonify({"error": "Employee not found"}), 404
    del employees[employee_id]
    return jsonify({
        "message": "Employee removed successfully"
    }), 200
if __name__ == '__main__':
    app.run(debug=True)
```

REST API Methods Documentation:

localhost:5000/students | New Request | p15_2.py | p15_3.py | New Request | BLACKBOX

POST http://127.0.0.1:5000/employees Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

JSON | XML | Text | Form | Form-encode | GraphQL | Binary

JSON Content Format

```
{
  "id": "1002",
  "name": "Sreeja",
  "salary": 100000
}
```

Status: 201 CREATED Size: 111 Bytes Time: 9 ms

Response | Headers 5 | Cookies | Results | Docs

```
{
  "employee": {
    "name": "Sreeja",
    "salary": 100000
  },
  "message": "Employee added successfully"
}
```

Response | Chart

PROBLEMS 28 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | AZURE | Python

localhost:5000/students | New Request | p15_2.py | p15_3.py | New Request | BLACKBOX

POST http://127.0.0.1:5000/employees Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

JSON | XML | Text | Form | Form-encode | GraphQL | Binary

JSON Content Format

```
{
  "id": "1003",
  "name": "Satwika",
  "salary": 1000000
}
```

Status: 201 CREATED Size: 113 Bytes Time: 10 ms

Response | Headers 5 | Cookies | Results | Docs

```
{
  "employee": {
    "name": "Satwika",
    "salary": 1000000
  },
  "message": "Employee added successfully"
}
```

Response | Chart

GET http://127.0.0.1:5000/employees Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "id": "1003",
3    "name": "Satwika",
4    "salary": 1000000
5  }
```

Status: 200 OK Size: 240 Bytes Time: 8 ms

Response Headers 5 Cookies Results Docs { }

```
1  {
2    "data": {
3      "101": {
4        "name": "John",
5        "salary": 50000
6      },
7      "1002": {
8        "name": "Sreeja",
9        "salary": 100000
10     },
11     "1003": {
12       "name": "Satwika",
13       "salary": 1000000
14     }
15   },
16   "success": true
17 }
```

PUT http://127.0.0.1:5000/employees/101/salary Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2
3    "salary": 70000
4  }
```

Status: 200 OK Size: 108 Bytes Time: 10 ms

Response Headers 5 Cookies Results Docs { }

```
1  {
2    "employee": {
3      "name": "John",
4      "salary": 70000
5    },
6    "message": "Salary updated successfully"
7  }
```

Pretty-print


```
{
  "data": {
    "1002": {
      "name": "Sreeja",
      "salary": 100000
    },
    "1003": {
      "name": "Satwika",
      "salary": 1000000
    },
    "101": {
      "name": "John",
      "salary": 70000
    }
  },
  "success": true
}
```

DELETE http://127.0.0.1:5000/employees/101 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2
3    "salary": 70000
4  }
```

After Delection Output:

Pretty-print


```
{
  "data": {
    "1002": {
      "name": "Sreeja",
      "salary": 100000
    },
    "1003": {
      "name": "Satwika",
      "salary": 1000000
    }
  },
  "success": true
}
```

Justification:

1. Manages employee records and salary updates via API.
2. Data model includes fields like id, name, department, salary.
3. PUT endpoint allows updating employee salary accurately.
4. JSON responses provide structured, auditable information.
5. AI generates docstrings, test cases, and validates data automatically.

Task 4 :Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

o GET /menu → List available dishes o POST /order

→ Place a new order o GET /order/{id} → Track

order status o PUT /order/{id} → Update an

existing order (e.g., change items) o DELETE

/order/{id} → Cancel an order

• AI should generate: o REST API code o Suggested

improvements (like authentication, pagination)

Expected Output:

• Fully working API simulating a food ordering backend.

Prompt:

Generate a RESTful API using Python Flask for an Online Food Ordering System.

Requirements:

- GET /menu → list all available dishes

- POST /order → place a new order

- GET /order/{id} → get order status

- PUT /order/{id} → update order (change items)

- DELETE /order/{id} → cancel an order

Use in-memory data structures (dictionary/list).

Return all responses in JSON format.

Include proper error handling.

Also suggest improvements like authentication and pagination.

Python Code:

```python
#Generate a RESTful API using Python Flask for an Online Food Ordering System.

#Requirements:
#- GET /menu → list all available dishes
#- POST /order → place a new order
#- GET /order/{id} → get order status
#- PUT /order/{id} → update order (change items)
#- DELETE /order/{id} → cancel an order

#Use in-memory data structures (dictionary/list).Return all responses in JSON format.Include proper error handling.Also suggest improvement
from flask import Flask, request, jsonify

app = Flask(__name__)

# Sample menu
menu = [
    {"id": 1, "name": "Pizza", "price": 250},
    {"id": 2, "name": "Burger", "price": 150},
    {"id": 3, "name": "Pasta", "price": 200}
]

# Orders storage
orders = {}
order_id = 1

# ✅ GET MENU
@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu)
```

```python
# ✅ GET MENU
@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu)

# ✅ PLACE ORDER
@app.route('/order', methods=['POST'])
def place_order():
    global order_id
    data = request.json

    if not data or 'items' not in data:
        return jsonify({"error": "Items required"}), 400

    order = {
        "id": order_id,
        "items": data["items"],
        "status": "Placed"
    }

    orders[order_id] = order
    order_id += 1

    return jsonify(order), 201

# ✅ GET ORDER
@app.route('/order/<int:id>', methods=['GET'])
def get_order(id):
    if id not in orders:
```

```python
# ✅ GET ORDER
@app.route('/order/<int:id>', methods=['GET'])
def get_order(id):
    if id not in orders:
        return jsonify({"error": "Order not found"}), 404

    return jsonify(orders[id])

# ✅ UPDATE ORDER
@app.route('/order/<int:id>', methods=['PUT'])
def update_order(id):
    if id not in orders:
        return jsonify({"error": "Order not found"}), 404

    data = request.json

    if 'items' in data:
        orders[id]['items'] = data['items']

    return jsonify(orders[id])

# ✅ DELETE ORDER
@app.route('/order/<int:id>', methods=['DELETE'])
def delete_order(id):
    if id not in orders:
        return jsonify({"error": "Order not found"}), 404

    orders[id]['status'] = "Cancelled"
    return jsonify({"message": "Order cancelled"})
```


```python
    return jsonify(orders[id])

# ✅ DELETE ORDER
@app.route('/order/<int:id>', methods=['DELETE'])
def delete_order(id):
    if id not in orders:
        return jsonify({"error": "Order not found"}), 404

    orders[id]['status'] = "Cancelled"
    return jsonify({"message": "Order cancelled"})
```


```python
# ✅ RUN SERVER
if __name__ == '__main__':
    app.run(debug=True)
```

REST API Methods Documentation:

equest | New Request | New Request | New Request | p4.py | New Request | loc BLACKBOX

POST http://127.0.0.1:5000/order Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

JSON | XML | Text | Form | Form-encode | GraphQL | Binary

JSON Content Format

```
1 {
2   "items": ["Biryani", "Chapathi"]
3 }
```

Status: 201 CREATED Size: 84 Bytes Time: 4 ms

Response | Headers 5 | Cookies | Results | Docs

```
1 {
2   "id": 2,
3   "items": [
4     "Biryani",
5     "Chapathi"
6   ],
7   "status": "Placed"
8 }
```

equest | New Request | New Request | New Request | p4.py | New Request | loc BLACKBOX

GET http://127.0.0.1:5000/menu Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

JSON | XML | Text | Form | Form-encode | GraphQL | Binary

JSON Content Format

```
1 {
2   "items": ["Biryani", "Chapathi"]
3 }
```

Status: 200 OK Size: 184 Bytes Time: 8 ms

Response | Headers 5 | Cookies | Results | Docs

```
1  [
2    {
3      "id": 1,
4      "name": "Pizza",
5      "price": 250
6    },
7    {
8      "id": 2,
9      "name": "Burger",
10     "price": 150
11   },
12   {
13     "id": 3,
14     "name": "Pasta",
15     "price": 200
16   }
17 ]
```

Response | Chart

THUNDER CLIENT

New Request

Activity | Collections | Env

filter activity

- GET 127.0.0.1:5000/order — just now
- DEL 127.0.0.1:5000/employees — 3 hours ago
- POST 127.0.0.1:5000/employees — 3 hours ago
- PUT POST 127.0.0.1:5000/books — 3 hours ago
- DEL 127.0.0.1:5000/students — 5 hours ago
- POST localhost:3000/orders — 7 mins ago
- PUT localhost:3000/students/69983bc0312... — 1 month ago
- POST localhost:3000/students — 1 month ago
- PUT localhost:3000/students/2

GET http://127.0.0.1:5000/order/1 Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

JSON | XML | Text | Form | Form-encode | GraphQL | Binary

JSON Content — Format

```
1 {
2   "items": ["Pizza", "Burger"]
3 }
```

Status: 200 OK Size: 84 Bytes Time: 4 ms

Response | Headers 5 | Cookies | Results | Docs

```
1 {
2   "id": 1,
3   "items": [
4     "Biryani",
5     "Chapathi"
6   ],
7   "status": "Placed"
8 }
```

Response | Chart

17 11 BLACKBOX Agent Open Website Update is ready, click to restart. Go Live Ready BLACKBOXAI: Open Chat Go Live

DELETE http://127.0.0.1:5000/order/1 Send

Query | Headers 2 | Auth | Body 1 | Tests | Pre Run

JSON | XML | Text | Form | Form-encode | GraphQL | Binary

JSON Content — Format

```
1 {
2   "items": ["Pizza", "Burger"]
3 }
```

Status: 200 OK Size: 35 Bytes Time: 11 ms

Response | Headers 5 | Cookies | Results | Docs

```
1 {
2   "message": "Order cancelled"
3 }
```

PUT http://127.0.0.1:5000/order/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "items": ["Tea", "Burger"]
3 }
```

Status: 200 OK Size: 81 Bytes Time: 20 ms

Response Headers 5 Cookies Results Docs { } ☰

```
1 {
2   "id": 1,
3   "items": [
4     "Tea",
5     "Burger"
6   ],
7   "status": "Cancelled"
8 }
```

Response Chart

Output:

Pretty-print


```
[
  {
    "id": 1,
    "name": "Pizza",
    "price": 250
  },
  {
    "id": 2,
    "name": "Burger",
    "price": 150
  },
  {
    "id": 3,
    "name": "Pasta",
    "price": 200
  }
]
```

Justification:

1. Simulates a real-time food ordering system with CRUD operations.
2. Endpoints handle menu listing, order placement, updates, and tracking.
3. JSON ensures seamless communication with front-end apps.
4. AI suggests improvements like authentication, pagination, and order tracking.
5. Demonstrates practical application of REST APIs in e-commerce scenarios.